

8. Skriv om labb 1 med exceptions så att programmet inte kraschar om man skriver in något fel.

Kapitel 6

Sekvenser och generatorer

6.1 Sekvenser

Python har stöd för ett antal olika sekvenstyper. Sekvenstyper i Python har gemensamt några vanliga operationer, t.ex. gäller:

- `+`: konkatenering/sammanslagning
- `*`: repetition
- `in` och `not in`: test om ett värde finns/inte finns som element i sekvensen
- `len()`: längden av sekvensen/hur många element
- `min()` och `max()`: minimum och maximum av element (om element-typerna är kompatibla med `<` och `>`)
- `[ : ]`: slicing för åtkomst av element
- m.fl.

Många av operationerna ovan känns förmodligen igen från `list`. Mycket riktigt är listor en av sekvenstyperna i Python. De andra typerna vi kommer att titta på i detta kapitel är tupler, range och strängar, vilka vi alla stött på tidigare i ett fall eller annat.

Sekvenstyper kan delas in i två underkategorier: *muterbara* och *icke-muterbara*. Med muterbarhet så menas att ett skapat objekt kan *ändras* på efter att det skapats. På samma sätt så är icke-muterbara objekt konstanta. Hur ett objekt används, och vilka potentiella sidoeffekter kod kan ha, kan bero på om ett objekt är muterbart eller inte.

För mer detaljerad information, se Python-dokumentationen om [sekvenstyper](#).

6.1.1 Listor

Listor är en sekvenstyp som är *muterbar*. Det är en viktig egenskap hos listor som betyder att vi kan uppdatera deras värden utan att kopiera listan. Vi har redan sett många exempel på vad som kan göras med listor, men för att poängtera vilken effekt det kan ha att listor är muterbara, betrakta på följande funktionsdefinition och kod:

```
def sum_of_increment(l, x):  
    for i in range(len(l)):  
        l[i] += x
```

```

ret_sum = sum(1)
return ret_sum

mylist = [3, 3, 4, 5]
print("mylist is:", mylist)
s = sum_of_increment(mylist, 2)
print("mylist after first call:", mylist)
print("First call returns:", s)
s = sum_of_increment(mylist, 2)
print("mylist after second call:", mylist)
print("Second call returns:", s)

```

```

mylist is: [3, 3, 4, 5]
mylist after first call: [5, 5, 6, 7]
First call returns: 23
mylist after second call: [7, 7, 8, 9]
Second call returns: 31

```

Resultatet blir olika varje gång funktionen kallas, även om funktionsanropet görs med samma variabler! Det här beror på att `mylist` ändras inuti funktionen, vilket går att göra eftersom listor är muterbara. Det här är också möjligt för att Python inte kopierar hela listan till funktionsargumentet (i det här fallet 1). Detta kan ibland vara användbart, men man bör vara medveten om och försiktig med detta, så att det inte får oanade konsekvenser.

För mer om listor och vilka funktioner som list-typen har, se Python-dokumentationen och [mer info om listor](#).

6.1.2 Tupler

Tupler liknar listor, men de är *icke-muterbara*. Så vi kan inte bara uppdatera en tupel som vi vill. Vi har sett tupler förr, även om vi kanske inte explicit skapat variabler av typen tupler: [multipla returvärden](#) från *funktioner* returneras nämligen som en tupel med flera element. Syntaxen för tupler är väldigt likt den som listor, med den skillnaden är att man använder `()` för att skapa en tupel istället för `[]`.

```

mytuple = (3, 3, 4, 5)
myothertuple = 3, 3, 4, 5 # equivalent syntax
print(mytuple)
print(mytuple == myothertuple)
print(mytuple[0])
print(mytuple[3:1:-1])
print(len(mytuple))
print(3 in mytuple)

# function returning multiple values
def integer_div(nominator, denominator):
    q = nominator // denominator
    r = nominator % denominator
    return q, r

divs = integer_div(17, 10)

```

```

print(type(divs))

# tuples are immutable
mytuple[0] = 42

(3, 3, 4, 5)
True
3
(5, 4)
4
True
<class 'tuple'>
TypeError: 'tuple' object does not support item assignment

```

Lägg märke till att slicing av en tupel ger tillbaka en tupel: slicing av en sekvenstyp ger tillbaka samma typ.

Om vi t.ex. skulle köra om kodexemplet från sektionen [Listor](#), med funktionen `sum_of_increment` definierad på samma sätt, fast med en tupel:

```

mytuple = (3, 3, 4, 5)
s = sum_of_increment(mytuple, 2)

TypeError: 'tuple' object does not support item assignment

```

Precis som listor, kan tupler innehålla olika datatyper och vi kan nästla dem:

```

t = (12345, 54321, 'hello!')
mytuple = (t, (1, 2, 3, 4, 5))
print(mytuple)

((12345, 54321, 'hello!'), (1, 2, 3, 4, 5))

```

Tupler kan innehålla *muterbara* objekt:

```

t2 = ([1, 2, 3], [3, 2, 1])
print(t2)

([1, 2, 3], [3, 2, 1])

```

Det sista exemplet `t2`, kan *inte* användas som nyckel i en uppslagstabell. Jämför t.ex.:

```

d = {t: 'hello'} # works
d2 = {t2: 'goodbye'} # TypeError: unhashable type: 'list'

```

Det här är som diskuterats tidigare för att nyckelvärden måste vara konstanta. I detta fall räcker det alltså inte med att själva tupeln är icke-muterbar eftersom den innehåller muterbara objekt (listor), vilket gör att `t2` *inte* kan användas som nyckel.

```

# t2[0] = [1, 2, 4] # throws TypeError, immutable
print(t2)
t2[0][0] = 42
print(t2)

```

```
([1, 2, 3], [3, 2, 1])  
([42, 2, 3], [3, 2, 1])
```

6.1.3 range

Pythons range är också en *icke-muterbar* sekvenstyp. Den beter sig som en tupel, men den har inte stöd för vissa operationer, t.ex. + och *.

```
myrange = range(10)  
print(myrange[3])  
print(myrange[2:9:3])  
print(4 in myrange)  
print(42 in myrange)  
myrange + myrange
```

```
3  
range(2, 9, 3)  
True  
False  
TypeError: unsupported operand type(s) for +: 'range' and 'range'
```

Återigen, slicing ger här tillbaka en range-typ.

6.1.4 Strängar

Strängar är även de ett exempel på en sekvenstyp, också *icke-muterbara*. Vi har precis som listor använt strängar flitigt. Det finns väldigt många användbara strängfunktioner; de flesta funktionerna har namn som tydligt säger vad de gör. Här nedan ges bara några exempel för vanligt förekommande strängfunktioner:

```
my_string = "Hello world! This is fun"  
print(my_string.index("o"))  
print(my_string.count("l"))  
print(my_string.upper())  
print(my_string.lower())  
print(my_string.capitalize())  
print(my_string[3:7])  
print(my_string[3:7:2])  
print(my_string[::-1])  
print(my_string.startswith("Hello"))  
print(my_string.endswith("asdfasdfasdf"))  
print(my_string.split())  
print(my_string.split("i"))
```

```
4  
3  
HELLO WORLD! THIS IS FUN  
hello world! this is fun  
Hello world! this is fun  
lo w  
l
```

```
nuf si sihT !dlrow olleH
True
False
['Hello', 'world!', 'This', 'is', 'fun']
['Hello world! Th', 's ', 's fun']
H
```

Glöm inte att man i Spyder lätt i t.ex. konsolen kan se vilka funktioner som är associerade med en sträng (eller generellt ett givet objekt) genom att använda sig av `dir()`. T.ex. kan man:

```
dir(my_string)
```

Detta kan ge lite “för mycket” information. Vill man ha en mer interaktivt sätt, kan man i konsolen skriva `my_string`. (lägg märke till punkten) och sen trycka TAB.

För mer om strängar och fler strängfunktioner, se [Python-dokumentationen](#).

6.2 Generatorer

Generatorer i Python är itererbara objekt som i varje iteration lämnar tillbaka ett värde till användaren. Vad värdet är beror på hur en generator har definierats. Vi har stött på flera itererbara objekt tidigare, t.ex. är alla sekvenstyper som precis beskrivits i [Sekvenser](#) itererbara.

Den största skillnaden mellan generatorer och sekvenstyper (förutom `range`) är när själva beräkningen av elementen sker: detta kan antingen ske *direkt* när man skapar objektet eller först när värdet på elementet *behövs*, d.v.s. när man väl utför en loop. Generatorer är ett exempel på en typ där värdet av ett element beräknas först när det behövs, detta är så kallad *lat evaluering*.

6.2.1 Lat och strikt evaluering

Om alla värden i ett objekt beräknas när objektet skapas kallar man det *strikt* eller *ivrig evaluering/beräkning* (eng. *eager evaluation*). Motsatsen är *lat* eller *selektiv evaluering* (eng. *lazy evaluation*) där värden beräknas när de behövs. Ett exempel med sekvenstyper som visar skillnaden:

```
mylist = [0, 1, 2, 3, 4, 5]
myrange = range(6)
print("mylist: ", mylist)
print("myrange: ", myrange)
```

```
mylist: [0, 1, 2, 3, 4, 5]
myrange: range(0, 6)
```

Här har vi skapat en lista `mylist`, som är ett exempel på strikt evaluering: alla värden som listan innehåller är här beräknade och sparade i minnet som utskriften visar. Utskriften av `range` å andra sidan visar bara just det, att det är ett objekt av typen `range` skapat med inparametrar `0, 6`. `range` är ett exempel på lat evaluering: alla värden som `myrange` innehåller *beräknas* varje gång man t.ex. gör `myrange[0]`. Lat evaluering kommer väl till hands när man använder sig av iteration, då det är mycket mer *minneseffektivt* än att iterera över ett objekt som är helt sparad i minnet.

`range` är lite av ett specialfall jämfört med de andra sekvenstyperna, just eftersom det är lat evaluering. Den finns för att det är en så vanlig struktur att ha en sekvens av ökande tal. Om man vill använda sig av lat evaluering, fast för mer komplicerade strukturer, så kommer generatorer väl till hands.

6.2.2 Generatorfunktioner

I Python kan man skapa en generatorfunktion på ett väldigt likt sätt som man skapar en vanlig funktion. När en vanlig funktion anropas, så exekveras all kod i funktionen fram tills att en `return`-sats stöts på, då ett värde (potentiellt) lämnas tillbaka till användaren och avslutar funktionskörningen. En generatorfunktion returnerar ett värde med `yield`, men funktionens *tillstånd sparas*. Nästa gång funktionen anropas så fortsätter funktionen från där den var sist tills *nästa* gång en `yield`-sats stöts på, vid vilken den lämnar tillbaka värdet och “pausar”.

Vi använder här funktionsanrop för generatorfunktioner lite löst. Egentligen kallas själva generatorfunktionen endast *en gång* och ger då tillbaka en generator. Det är denna generator som när den itereras över varje gång exekverar koden i generatorfunktionen.

Ett exempel visar nog detta bättre:

```
def inf_gen(start=0, step_size=1):
    """Infinitely generate ascending numbers"""
    while True:
        yield start
        start += step_size
```

Detta är en oändlig uppräknings av tal: en “oändlig range”. Hade vi försökt att använda oss av en lista, så inser man snabbt att det är svårt att skapa ett objekt med *oändligt* många element: vi hade fått slut på minne på datorn. Men representeras det som en generator är detta inget problem och vi kan fortfarande iterera över den. För att komma åt underliggande generator som man kan iterera över anropar man generatorfunktionen:

```
my_inf_gen = inf_gen(10, 1)
print(my_inf_gen)
print(next(my_inf_gen))
print(next(my_inf_gen))
print(next(my_inf_gen))
```

```
<generator object inf_gen at 0x7f1d7855be40>
10
11
12
```

Här har vi använt oss av `next`-funktionen med generatoren `my_inf_gen` som argument för att stega fram nästa värde. Det är faktiskt implicit `next` som anropas om man skulle iterera över `my_inf_gen` genom att t.ex. göra `for i in my_inf_gen: print(i)`. Vi kan inte göra det här, eftersom det skulle fortsätta i oändlighet, men testa gärna själv för att se vad som händer. (För att avsluta en oändlig loop kan man trycka CTRL+c)

Generators behöver självklart inte vara oändligt itererbara (som ovan). Som du ser kan du bestämma själv hur komplicerad en generator ska bli med nästan samma syntax som för en vanlig funktion. Iterationen över en generator avslutas när funktionskroppen körs utan att stöta på ett `yield`. Precis som man kan ha flera `return`-satser i en funktion, kan man ha flera `yield` i en generatorfunktion. Skillnaden är att funktionsblocket fortsätter från där det slutade vid senaste `yield`. En viktig detalj, och skillnad mot `range`, är att en generator bara kan itereras över *en gång*.

Vi illustrerar ovan koncept med en enkel generatorfunktion:

```
def my_generator(stop=5):
    i = 0
    j = 0
    while i + j < stop:
        print("i: ", i, "j:", j)
        i += 1
        yield i + j
        print("i: ", i, "j:", j)
        i += 1
        j += 1
        yield i + j
        print("i: ", i, "j:", j)

it = 1
gen = my_generator()
for i in gen:
    print("Iteration: ", it, "gen returns: ", i)
    it += 1

# loop a second time
for i in gen:
    print("Will this be printed??")
```

```
i: 0 j: 0
Iteration: 1 gen returns: 1
i: 1 j: 0
Iteration: 2 gen returns: 3
i: 2 j: 1
i: 2 j: 1
Iteration: 3 gen returns: 4
i: 3 j: 1
Iteration: 4 gen returns: 6
i: 4 j: 2
```

6.2.3 Generatoruttryck

Generatorer kan även skrivas på ett liknande sätt som listomfattningar. Det skapas då ingen lista utan istället får man en generator som man kan arbeta vidare med, framförallt iterera över med olika verktyg. Här skapas en lista och en generator.

```
big_list = [x**2 for x in range(10**8)]
big_generator = (x**2 for x in range(10**8))
print("Size of big_list in memory:", big_list.__sizeof__())
print("Size of big_generator in memory:", big_generator.__sizeof__())
```

```
Size of big_list in memory: 859724448
Size of big_generator in memory: 96
```

Som vi ser har listomfattningen genererat hela listan i minnet på datorn. Detta illustrerar vikten av att använda sig utav generatorer när vi ska iterera över många värden.

6.2.4 Exempel: Generator för alla primtal

Generatorer är smidiga att använda till vissa saker, bland annat oändliga sekvenser.

Låt oss säga att vi vill beräkna alla primtalsdelare till ett tal x . Vi kan då skapa en generator med alla primtal och se hur många gånger varje primtal delar x :

```
def is_prime(x):
    for t in range(2, x):
        if x % t == 0:
            return False
    return True

def prime_divs(x):
    primes = (i for i in inf_gen(2) if is_prime(i))
    ps = []
    for p in primes:
        if p > x:
            return ps
        else:
            while x % p == 0:
                ps.append(p)
                x //= p

print(prime_divs(2))
print(prime_divs(3))
print(prime_divs(24))
print(prime_divs(25))
print(prime_divs(232))
```

```
[2]
[3]
[2, 2, 2, 3]
[5, 5]
[2, 2, 2, 29]
```

6.3 Uppgifter

1. Vi såg att funktionen `sum_of_increment(1, x)` ändrade i vår lista när vi kallade på den flera gånger. Vi såg även att vi inte kunde kalla på `sum_of_increment(1, x)` med en tupel. Kan du av detta dra slutsats när det kan vara bra att använda sig av tupler?
2. Skriv en funktion `is_palindrome` som tar en sträng som input, och returnerar `True` om strängen är ett palindrom (d.v.s. om det är samma sak framlänges som baklänges) och returnerar den omvända strängen om det inte är ett palindrom.

Tips: använd sträng-slicing.

3. Fungerar `is_palindrome` som du skrev i uppgift 2 för listor och tupler? Varför?
4. Skriv en funktion `occurrences` som beräknar hur många gånger bokstäver förekommer i en

sträng. Funktionen ska returnera en uppslagstabell med bokstäverna som nycklar och antal gånger bokstäverna förekommer i strängen. Funktionen ska inte göra skillnad på versaler och gemener, utan räkna dessa gemensamt (för en given bokstav). Alla tecken som *inte* är bokstäver ska räknas under nyckeln 'non_alphas', och tecken som *inte* finns med i strängen ska inte finnas med i uppslagstabellen.

Tips: använd dig av de inbyggda strängfunktionerna. Se till exempel Python-dokumentation för [strängfunktioner](#)

Exempel:

```
s = 'Hello my very happy friend! Is the sun shining?!'
uo = occurrences(s)
print(s)
for k, v in uo.items():
    print(k, ":", v)

s = 'JjJJJjj hHhh !/;:(+[//==*])'
uo = occurrences(s)
print("\n" + s)
for k, v in uo.items():
    print(k, ":", v)
```

```
Hello my very happy friend! Is the sun shining?!
h : 4
e : 4
l : 2
o : 1
non_alphas : 11
m : 1
y : 3
v : 1
r : 2
a : 1
p : 2
f : 1
i : 4
n : 4
d : 1
s : 3
t : 1
u : 1
g : 1

JjJJJjj hHhh !/;:(+[//==*])
j : 7
non_alphas : 17
h : 4
```

5. Använd `occurrences(s)` ovan för att skriva ett program som läser in en fil och skriver ut bokstäverna som nycklar och antalet gånger bokstäverna förekommer för hela filen. För att testa kan ni

använda filen `palindrome.txt` med innehåll:

```
not palindrome line
palindrome line enil emordnilap
no way
hello ! olleh
Tomorrow will be a glorious day!!!
oooooooooooooooooooooooooooo
rust is also a programming language
oooooooooooooooooooooooooooo
```

Tips: läs in hela filinnehållet som en sträng.

6. Skriv en generatorfunktion `fibonacci(stop=x)` som genererar Fibonacci tal (se <https://sv.wikipedia.org/wiki/Fibonaccital>) mindre eller lika med `x`.

Tips: Använd dig utav strukturen i funktionen `my_generator` ovan.

7. I första raden i funktionen `prime_divs` i [Exempel: generator för alla primtal](#) så använde vi oss av ett generatoruttryck för att "filtrera" alla tal ur en generator som var primtal i en ny generator. Man kan för sin filter-generator alltså skriva:

```
filter_gen = (i for i in iterable if function(i))
```

Här är `function` en (användardefinierad) funktion som ger tillbaka `True` eller `False` när `i` ges som argument, och `iterable` är ett itererbart objekt (t.ex. en generator som vi hade i `prime_divs`). Alla `i` som ger `function(i) == True` kommer vara en del av `filter_gen`. Python har redan implementerat detta i funktionen `filter`, som du kan läsa dokumentationen om. En liknande funktion är `map` (se dokumentationen) där generatoren som skapas bör innehålla *värdena* som en funktion `f` *returnerar* när `f` appliceras på alla element i ett itererbart objekt.

Skriv en egen funktion `map_gen` som åstadkommer detta genom att ta en *godtycklig funktion* som argument och ett itererbart objekt, och som *returnerar* en generator.

Exempel:

```
gen = map_gen(int, '123456')
sum(gen)
```

```
21
```

8. Skriv en funktion `palindrome_rows` som tar ett filnamn som argument, och kollar varje rad i filen om det är ett palindrom eller inte. `palindrome_rows` ska resultera i en generator som man kan iterera över när funktionen anropas.

Tips: tänk på att rader läses in *inklusive* ny-rad-tecken. Modifiera `is_palindrome` från uppgift 2 så att den bara lämnar tillbaka `True` eller `False` baserat på om strängen är ett palindrom (alltså inte lämnar tillbaka själva strängen baklänges om det *inte* är ett palindrom).

Exempel med filen `palindrome.txt` ovan:

```
p_rows = palindrome_rows('palindrome.txt')
i = 1
for p in p_rows:
    print("Row", i, "palindrome?", p)
    i += 1
```

```
Row 1 palindrome? False
Row 2 palindrome? True
Row 3 palindrome? False
Row 4 palindrome? True
Row 5 palindrome? False
Row 6 palindrome? True
Row 7 palindrome? False
Row 8 palindrome? True
```

9. Du “vet” att det i en fil `almost_lowercase.txt` som ska bestå av endast gemener och tecken gömmer sig *ett enda* versalt ord, men du vet inte vilket det är. Det här tycker du är extremt jobbigt! Använd dig av `open` tillsammans med strängfunktioner för att *utan en loop* ta reda på vilket ord det är. Du kan anta att ett “ord” avgränsas av mellanrum och att alla övriga ord (utöver det du letar efter) består endast av gemener och/eller tecken.

Tips: någon av `strip`-funktionerna tillsammans med indexering kan komma till användning här.

Exempel på `almost_lowercase.txt`:

```
this is a file that /should/ consist of only lowercase letters.
but it doesn't!
there is a hidden upppercase word right HERE that you are supposed to find.
```